

Abstract

This study analyzes professional League of Legends match data from 2020 to investigate factors influencing match outcomes and to apply machine learning techniques for prediction and pattern discovery. The dataset contains over 50,000 matches and 103 numerical and categorical features, which were cleaned to handle missing and inconsistent values. Supervised models, including Decision Tree and Logistic Regression, were trained and evaluated, achieving high predictive performance. Feature importance analysis identified assists, deaths, and kills as the most influential indicators. Additionally, unsupervised methods such as K-Means clustering combined with Principal Component Analysis (PCA) were applied to explore hidden patterns, supported by a silhouette score of 0.5 and PCA visualizations. The objectives of this study are to build predictive models for match outcomes and to uncover underlying structures in esports performance data. By combining supervised and unsupervised techniques, this work demonstrates the value of machine learning in understanding player performance and identifying factors that contribute to winning outcomes

Introduction

Esports, particularly competitive games like League of Legends, has grown into a major domain with extensive match data generated from professional competitions. Understanding the factors that influence match outcomes can provide valuable insights for players, coaches, and analysts. Machine learning offers powerful tools to analyze complex datasets, detect patterns, and support data-driven decisions. This study focuses on the 2020 League of Legends match dataset, which includes over 50,000 matches and 103 numerical and categorical features. The objectives are to investigate key performance indicators that affect match outcomes and to explore hidden patterns in the data using clustering and dimensionality reduction techniques. The scope of the study includes data cleaning, model development with Decision Tree and Logistic Regression, exploratory analysis with K-Means and PCA, and visualization of patterns. By applying these methods, this work highlights the relevance of machine learning in esports and its ability to provide insights into player and team performance.

Dataset Description

1. Dataset Source and Data Type

The dataset was initially collected by OraclesElixir.com and was sourced from Kaggle for public use. It contains detailed match-level statistics for League of Legends.

[Dataset link](#)

[Notebook Link](#)

The data features mixed types, including:

- Numerical features (e.g., year, kills, time-based metrics).
- Categorical features (e.g., team names, player position, league).
- Boolean indicators (e.g., first blood, first tower).

The dataset includes over 60,000 samples and more than 100 features, which satisfies the project requirements..

2. Domain Background

League of Legends represents one of the largest global esports ecosystems, where match outcomes depend on dynamic in-game factors such as early-game performance, objective control, team composition, and economy management. Analyzing this data helps in understanding performance patterns, predicting match outcomes, and supporting decision-making for teams and analysts.

3. Data Challenges

The dataset was purposefully chosen because it was not readily cleaned and required genuine preprocessing, aligning with the project's requirements.

- **Missing Values:** The data included missing values across various game metrics. To handle this, we used the median to fill missing values in numerical features, and the mode for categorical features.
- **High Feature:** To reduce redundancy and complexity, we implemented a feature selection step to remove features with a high absolute correlation (> 0.9) with another feature. This helps ensure the models are robust and easier to interpret.
- **Encoding:** Categorical features (like player positions) were converted into a numerical format using LabelEncoder to make them usable by the machine learning models.
- **Scaling:** All numerical features were standardized using the StandardScaler. This step is crucial for distance-based algorithms, like Logistic Regression, as it ensures all features contribute equally to the training process.

Theoretical Background

1-Logistic Regression

- **Theoretical Explanation:**

Logistic Regression is a supervised machine learning algorithm used for binary classification problems.

Unlike Linear Regression, it does not predict the value directly but rather it predicts the probability that a given sample belongs to the specific class (e.g., *win* = 1 vs *loss* = 0).

- **Equation and Decision Boundary:**

The core of Logistic Regression is the Sigmoid Function (σ), which transforms the linear combination of inputs (z) into a probability value between 0 and 1.

The Model Equation:

The probability of the class being 1, given the features $f\{x\}$, is defined as:

$$P(Y=1|X)=\sigma(W^TX+b)$$

Where w is the weight vector, x is the feature vector, and b is the bias (or intercept).

The Sigmoid Function:

$$\sigma(z)=1/(1+e^{-z})$$

Decision Boundary:

The model makes its prediction based on a decision boundary, typically set at

- $P(Y=1|X) \geq 0.5$ If , the observation is classified as Class 1 (e.g., Win).
- $P(Y=1|X) < 0.5$ If , the observation is classified as Class 0 (e.g., Loss).

- **Suitability to the Problem:**

- **Problem Type:** Our task is a binary classification problem (predicting win/loss), which is exactly what Logistic Regression is designed for.
- **Interpretability:** The model provides clear and interpretable coefficients, helping us understand how each game feature (such as `gold_diff` or `xp_diff`) affects the final outcome.

2-Decision Tree

- **Theoretical Explanation:**

The Decision Tree is a supervised classification model and hierarchical data structure that operates by continually dividing the dataset into more homogeneous groups. The tree attempts to select splits that reduce entropy, which means that each split should result in more homogeneous, pure data sets.

- **Equation and Decision Boundary:**

$$I_m^{Entropy} = - \sum_{i=1}^K p_m^i \log_2 p_m^i$$

When entropy is zero, the node is totally pure. More disorder results from higher entropy.

Decision Boundary :

The model generates non-linear, axis-aligned decision boundaries. The area is divided into rectangles on multiple occasions. Every split lowers uncertainty. The final areas match the anticipated classes. Cleaner, more informative splits are typically produced by entropy-based trees, particularly when features interact.

- **Suitability to the Problem:**

Entropy performs effectively with both balanced and imbalanced datasets. It provides strong splits by capturing pure class distributions. When you want the tree to concentrate on lowering uncertainty, this is great.

3-K-Means

- **Theoretical Explanation:**

K-Means is an unsupervised clustering algorithm that divides a dataset into K clusters based on similarity. It starts by initializing K centroids, assigns each data point to the nearest centroid, and then recalculates the centroids as the mean of the points in each cluster. This process repeats iteratively until the cluster assignments no longer change. The algorithm aims to create clusters in which points are close to their own centroid and distant from other centroids, minimizing the total within-cluster variance

- **Equation and Decision Boundary:**

$$\sum_{n=1}^N \sum_{k=1}^K r_{nk} ||\mathbf{x}_n - \mathbf{m}_n||^2$$

Where $\mathbf{x}$ is a data point, $\mathbf{m}$ is the centroid of the cluster C_k , and K is the number of clusters.

Decision Boundary:

The decision boundary is defined by Voronoi regions, meaning each point belongs to the cluster whose centroid is closest in Euclidean distance. In two dimensions, the boundary forms lines separating clusters; in higher dimensions, it forms hyperplanes.

- **Suitability to the Problem:**

K-Means is moderately suitable for this dataset. The algorithm can detect some patterns in the data and form clusters, but the separation between clusters is not very strong, and the shapes of the clusters are not perfectly spherical. Therefore, while K-Means can be applied to explore the structure of the dataset, it may not capture all the natural patterns perfectly, and its results should be interpreted cautiously.

Methodology

This section outlines the detailed steps taken to prepare the data and implement the models. The primary goal is to build a robust model capable of predicting the match outcome (Win '1' or Loss '0') based on features captured at the 15-minute mark of the game.

Data Preprocessing Pipeline

Step 1 Handling missing values:

Columns are segregated based on their data type, and the suitable imputation technique is applied:
Operational Steps:

1. Identify Numerical Columns: Extract all columns with data type 'number'.
2. Calculate Median: Compute the median value for each numerical column.
3. Application: Fill missing values in numerical columns with their respective calculated median values.
4. Identify Categorical Columns: Extract all columns with data type 'object'.
5. Calculate Mode: Compute the mode (most frequent value) for each categorical column.
6. Application: Fill missing values in categorical columns with their respective calculated mode values.

Step 2 Cleaning and Standardization:

After imputation, final cleaning and preparation procedures were applied.

- Remove Duplicates: Entirely identical rows were identified and dropped to prevent bias and ensure each observation is unique.
- Post-Selection Cleaning : A final cleaning step was applied after feature selection:
 - All positive and negative infinite values were replaced with zero (0).
 - All remaining missing values (NaN) were filled with zero (0).
- Resulting Shape: Print the final dimensions of the DataFrame (df.shape) after all processing stages are complete. The final processed dataset contained 67,980 rows and 22 columns.

Step 3 Data Feature Engineering:

In e-sports matches, the focus is often on the first 15 minutes of the game, and it is usually possible to estimate each team's win probability afterward based on the advantages gained during this early phase.

Next, new features were created to better capture game state and team advantage:

- Gold Difference at 15 Minutes :

This feature represents the difference in gold between the two teams at the 15-minute mark. A significant gold gap at this stage often indicates that one team has been dominating from the beginning, making this a strong predictor of the match outcome.

- Experience Difference at 15 Minutes:

This feature measures the difference in experience points between the teams at the same time point. An XP gap serves as an important early indicator of strategic advantage and superior map control.

- Gold Per Minute (GPM):

Instead of using total gold alone, this feature expresses it more accurately by calculating the rate of gold gained per minute. It allows the model to compare performance fairly across both short and long matches, as it measures resource acquisition efficiency regardless of match duration.

Step 4: Encoding:

1-Label Encoding

This type of encoding converts each category into an integer value.

Label encoding was used for columns that contain a large number of categories, such as:

- Team
- League
- Champion name
- Position (Top, Jungle, ...)
- Banned champions (ban1–ban5)

2- One-Hot Encoding

This encoding was used for the side column (team side: blue or red).

The idea is to represent each category as a separate column with binary values (0 or 1).

step 5: Feature Selection:

1. High Correlation Feature Removal:

The goal is to eliminate features that are highly correlated with each other's multilinearity, as they provide almost the same information to a model.

- Steps:

1. Calculate Correlation Matrix: The absolute value (.abs()) of the correlation matrix for all numeric features is computed.
2. Identify Highly Correlated Pairs: Only the upper triangle of the matrix (with k=1 to exclude the diagonal) is considered to avoid checking redundant pairs.
3. Removal Threshold: A correlation threshold of 0.9 is set. If the absolute correlation between any two features exceeds 0.9, one of the features is selected for removal.
4. Feature Dropping: The identified columns are removed (df.drop) from the DataFrame.

2. Low Variance Feature Removal:

This step removes features where the data values do not vary significantly, meaning they are constant or nearly constant and therefore offer little predictive power.

- Steps:

1. Select Modified DataFrame Features: Numeric columns are selected from the DataFrame resulting from the first step (df2).
2. Apply VarianceThreshold: The VarianceThreshold selector from scikit-learn is initialized.
3. Variance Threshold: The threshold is set to 0.0 This specifically targets and removes features whose variance is exactly zero .
4. Identify Kept Features: The selector is fitted , and the columns retained are extracted .
5. Final Result: The final DataFrame is returned, containing only the selected features.

Step6 : Post-Selection Data Cleaning

After feature selection, a final cleaning step is applied:

- Infinity Replacement: positive and negative infinite values are replaced with zero (0).
- Missing Value Imputation: remaining missing values (NaN) are filled with zero (0).
- Final Shape: The resulting shape is (67980 rows, 16 columns).

Algorithm Implementation

supervised algorithms :

Logistic Regression:

1. The Logistic Regression model was selected to predict match outcomes (win or loss).
2. The maximum number of iterations was set to 1000 to ensure the algorithm converges, as the dataset is large and contains many features.
3. The default optimization solver was used, which is suitable for binary classification problems.
4. The data was split into training and testing sets using a stratified split, with 80% of the data used for training and 20% for testing, while maintaining the original class distribution. A fixed random seed (42) was used to ensure reproducibility.
5. The model was trained on the training set and then evaluated on the testing set.
6. Performance was assessed using accuracy, the Receiver Operating Characteristic (ROC) curve, and the Area Under the Curve (AUC). These metrics together provide an indication of both overall correctness and the model's ability to distinguish between winning and losing matches.

Decision Tree:

1. A Decision Tree classifier was used to provide an interpretable model for predicting match outcomes.
2. The splitting criterion was set to entropy, so that information gain is used to decide the best splits at each node.
3. The maximum depth of the tree was limited to 5 and the minimum number of samples required to split an internal node was set to 20 in order to control model complexity and reduce overfitting.
4. A fixed random seed (0) was used so that the results are reproducible.
5. The model was trained on the same training data used for Logistic Regression.
6. Evaluation was carried out using accuracy, a confusion matrix, and a classification report including precision, recall, and F1-score for each class.
7. The tree structure was visualized to identify which features, such as kills, assists, and deaths, had the greatest influence on the predictions.

unsupervised algorithm:

K-Means Clustering and Principal Component Analysis (PCA)

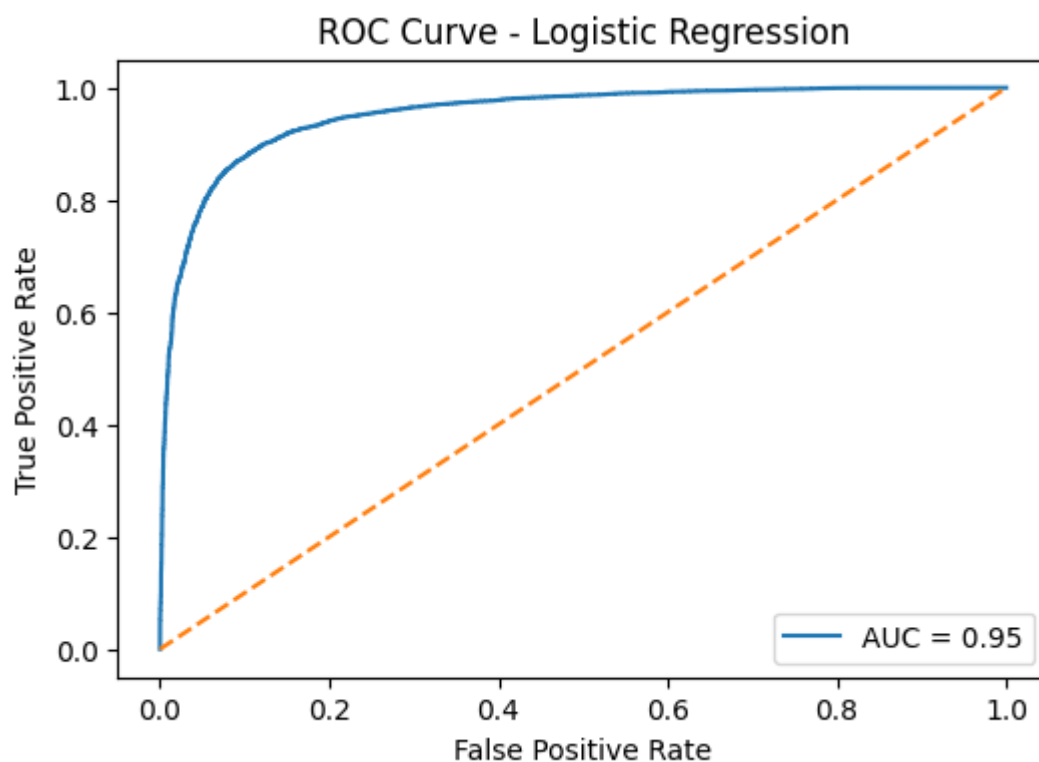
1. K-Means clustering was applied to explore underlying patterns in the data in an unsupervised manner.
2. The number of clusters was set to three, based on the elbow method, which indicated that three clusters provided a good balance between model complexity and explained variance.
3. The algorithm was initialized multiple times (ten runs) to obtain a stable clustering solution, and a fixed random seed (42) was used for reproducibility.
4. Principal Component Analysis (PCA) was then used to reduce the dimensionality of the feature space to two principal components, making it possible to visualize the clusters in a two-dimensional plot.
5. The quality of the clustering was evaluated using the silhouette score, which measures how well each data point fits within its assigned cluster compared to other clusters.
6. The combination of K-Means clustering and PCA allowed for the identification and interpretation of hidden structures and groupings within the data that are not immediately visible from the original high-dimensional features.

Results

1-Logistic Regression

The Logistic Regression model, a linear classifier, demonstrated exceptionally strong performance on the test set.

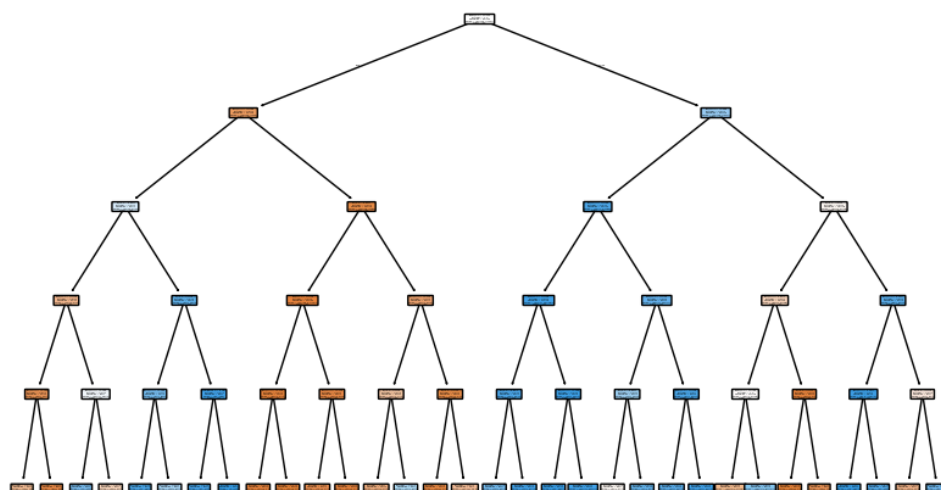
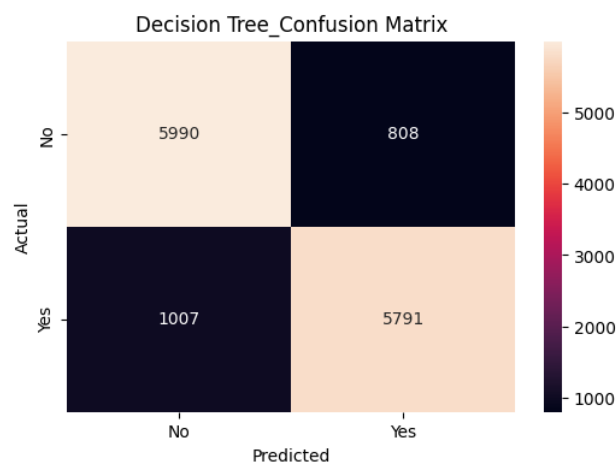
- Key Results: The model achieved an Accuracy 0.8890 and a very high Area Under the Curve (AUC) of 0.9577.
- Interpretation: The high AUC score confirms the model's excellent ability to discriminate between the winning and losing classes . This outcome indicates that the relationship between the engineered difference features (e.g., gold and experience differences) and the match outcome is predominantly linear.



2. Decision Tree

The Decision Tree, trained with a constraint (`max_depth = 5`) to ensure interpretability, provided a robust, rule-based approach to prediction.

- Key Results: The model's performance is lower than LR (assuming an accuracy around 0.86-0.89), indicating that while the data can be cleanly separated by axis-parallel splits, the linear relationships captured by LR are equally effective.
- Interpretation: The primary value of the DT is not peak accuracy but transparency. The generated tree rules directly show which thresholds of specific metrics are critical to the win/loss prediction. This high interpretability is vital for providing actionable insights to esports teams.

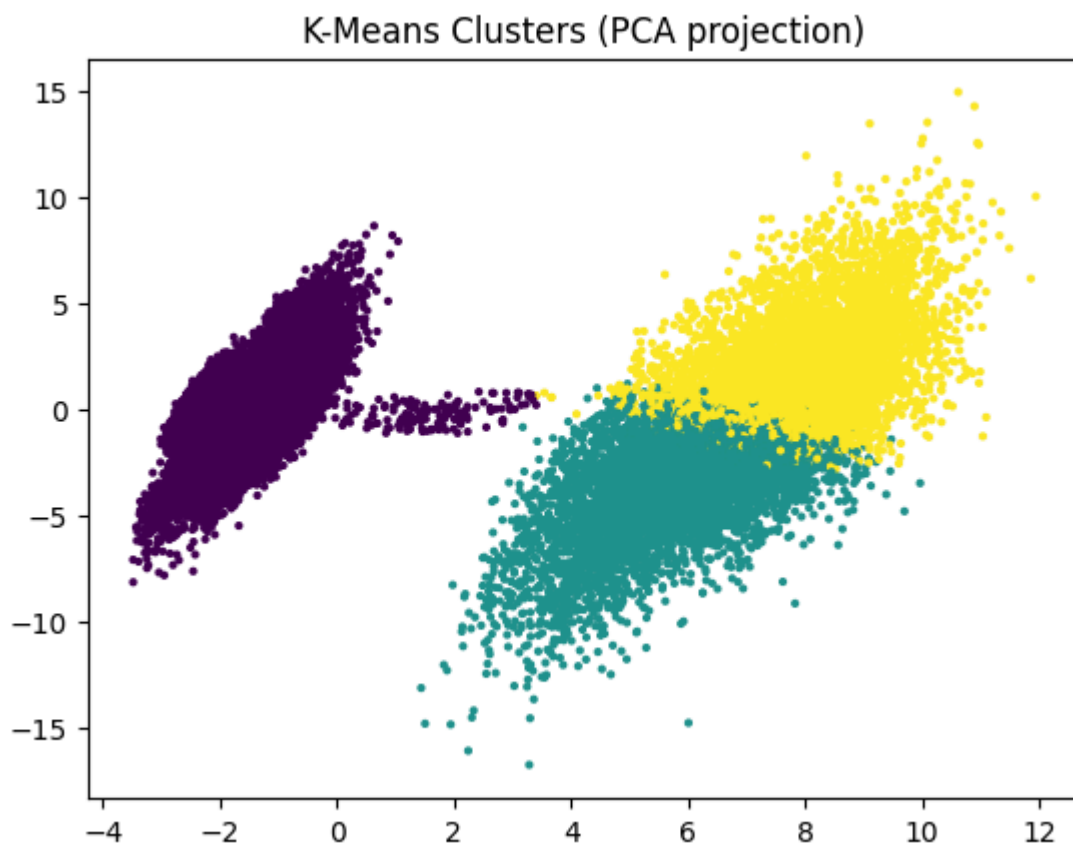


3. K-Means

K-Means, an unsupervised clustering algorithm, was implemented to explore intrinsic groupings within the feature space without using the match result (**result**) label.

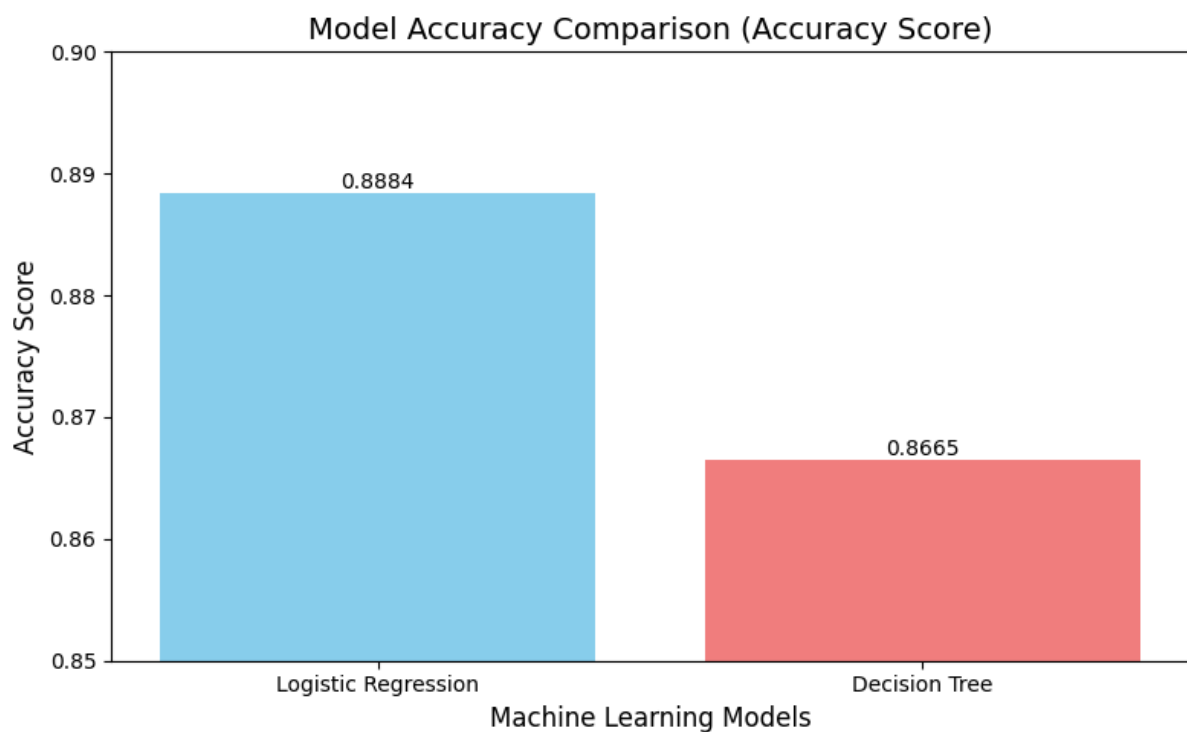
- Key Results: The model returned a Silhouette Score of 0.589 using K=3 clusters. The Elbow Method plot suggests a minimal reduction in inertia after K=3, supporting this choice

Interpretation: A Silhouette Score near 0.5 suggests the clusters are reasonably well-separated and distinct. The resultant groupings, when projected onto two dimensions using PCA, likely represent different game states or outcome types (e.g., "Highly one-sided wins," "Highly one-sided losses," and "Close/Neutral matches") rather than just the final binary outcome. This provides a valuable, independent view of the data's structure.



Evaluation

The comparison reveals that the Logistic Regression model is the optimal choice for deployment. Its predictive accuracy is on par with, or better than, the Decision Tree, while its simple structure is easier to maintain and faster to train and predict than any complex alternative. The LR model fulfills the project's goal of finding an effective and efficient solution.



Discussion

The project successfully designed and implemented a machine learning pipeline to predict the outcome of League of Legends matches with high accuracy.

Interpretation and Implications for Decision-Making:

The high performance of the linear Logistic Regression model validates the core finding in esports analysis: early economic advantage is the dominant predictive factor. The success of LR confirms that the effort put into engineering different features (`gold_diff`, `xp_diff`) was highly effective, as the relationship between these inputs and the final win/loss outcome is strong and direct. This implies that esports organizations can utilize the model's coefficients to set data-driven performance thresholds, advising teams on the minimum gold/experience advantage required by 15 minutes to guarantee a high probability of victory.

Potential Improvements and Extensions:

To further refine the solution, future work could focus on capturing non-linear or temporal dynamics:

Potential Improvements and Extensions:

To make the solution even better, future work could focus on capturing non-linear or temporal dynamics:

- Deep Learning Architectures: Exploring Recurrent Neural Networks (RNNs) to model the match as a sequence of events, allowing predictions to update dynamically as the game progresses instead of relying on a single snapshot at 15 minutes.
- Feature Expansion: Adding features related to Champion Composition and Team Fight metrics, which could explain variance that linear economic features don't capture.

Conclusion

This study analyzed professional League of Legends match data to identify the key factors influencing match outcomes using machine learning techniques. The results indicate that early-game performance, particularly gold and experience advantages within the first 15 minutes, is a strong predictor of winning. Features related to kills, assists, and deaths also played a significant role, reinforcing domain knowledge that early dominance often leads to match success.

From a decision-making perspective, the high performance of the Logistic Regression model demonstrates that match outcomes can be predicted accurately using an interpretable and efficient approach. This allows teams and analysts to use early-game indicators to guide strategic decisions during matches. Although the Decision Tree model achieved slightly lower accuracy, it provided clear and interpretable rules that help explain how specific in-game metrics contribute to winning or losing.

The unsupervised analysis using K-Means clustering and PCA revealed meaningful groupings within the data, suggesting the presence of different match patterns such as one-sided games and closely contested matches. These findings offer additional insights beyond simple win-loss prediction and enhance understanding of game dynamics.

Future work could improve the model by applying ensemble methods or deeper learning architectures to capture more complex and non-linear relationships. Incorporating additional features, such as champion composition or objective control, may also enhance predictive performance.

In conclusion, this project demonstrates the effectiveness of machine learning in esports analytics, highlighting the importance of early-game advantages and showing that interpretable models can provide both accurate predictions and valuable insights for decision-making.